Name: ______ Cohort/Batch: ______ Date: ______ Score: ______

NEO4J PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Databases

TOTAL: 10 QUESTIONS

Q1 What type of database is Neo4j and what are its core strengths? *Threat Model*

Q2 How does Neo4j guarantee consistency and handle transactions? *Architecture*

Q3 What index types does Neo4j support and when do you use each? *Configuration*

Q4 How do you design a schema or data model in Neo4j? *Hardening*

Q5 How does Neo4j scale horizontally? *SSO / IdP*

Q6 How do you monitor and tune slow queries in Neo4j? *Observability*

Q7 What backup and restore strategies does Neo4j support? *Lifecycle*

Q8 How do you handle schema migrations in Neo4j? *Compliance*

Q9 What security features does Neo4j provide? *Testing*

Q10 What are common anti-patterns when using Neo4j in production? *Tuning*

ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Neo4j is a relational, document, key-value, graph,

Mitigation

Neo4j is a relational, document, key-value, graph, or time-series store. Its strengths such as ACID guarantees, horizontal scale, or flexible schema guide the workloads it is best suited for.

A2 Neo4j provides either full ACID transactions or

Primitives

Neo4j provides either full ACID transactions or eventual consistency depending on its CAP theorem positioning. Transaction support varies from none (simple K/V stores) to serializable isolation (relational DBs).

A3 Neo4j offers B-tree, hash, full-text, spatial, or

Hardening

Neo4j offers B-tree, hash, full-text, spatial, or composite indexes depending on the engine. Choose based on query patterns: B-tree for range queries, hash for equality lookups, full-text for search.

A4 Schema design in Neo4j involves normalizing relations

Gotchas

Schema design in Neo4j involves normalizing relations (relational DB) or denormalizing for access patterns (document/key-value). Understanding query needs upfront prevents costly migrations later.

A5 Neo4j scales via replication (read replicas) for

Federation

Neo4j scales via replication (read replicas) for read-heavy workloads and sharding (partitioning data by key) for write-heavy ones. Some Neo4j implementations handle this automatically; others require manual configuration.

A6 Neo4j provides a slow query log, explain/explain

Observability

Neo4j provides a slow query log, explain/explain plan, or profiling tools to surface inefficient queries. Common fixes include adding indexes, rewriting queries, or increasing cache size.

A7 Neo4j supports logical backups (export SQL or

Automation

Neo4j supports logical backups (export SQL or BSON), physical backups (filesystem snapshot), and continuous replication to a standby. Point-in-time recovery requires WAL/oplog archiving on top of backups.

A8 Schema migrations in Neo4j are managed with

Governance

Schema migrations in Neo4j are managed with versioned migration files applied in order by a migration tool. Migrations should be idempotent and tested in staging before production to avoid downtime.

A9 Neo4j supports role-based access control, encrypted

Assessment

Neo4j supports role-based access control, encrypted connections (TLS), encryption at rest, and audit logging. Always restrict user privileges to the minimum needed and disable anonymous access in production.

A10 Common mistakes include selecting all columns instead

Optimization

Common mistakes include selecting all columns instead of needed fields, missing indexes on frequently queried columns, running migrations without backups, and storing large blobs directly in the database instead of object storage.